

Guia Essencial de Comandos Git e GitHub

Este documento detalha os fluxos de trabalho fundamentais utilizando Git e GitHub, estruturados para atender desde a implementação de novas funcionalidades (features) até a correção rápida de problemas críticos (hotfixes). Projetado com uma abordagem didática, pode ser utilizado tanto como referência rápida no dia a dia do desenvolvimento de software quanto como material de apoio educacional.

1. Fluxo Básico de Codificação (Feature Workflow)

O fluxo básico é utilizado no desenvolvimento diário para adicionar novas funcionalidades de forma isolada, garantindo que o código da ramificação principal permaneça estável durante todo o processo.

- 1. Atualizar o repositório local:** Antes de iniciar qualquer trabalho, garanta que sua base de código local está sincronizada com a versão mais recente do servidor remoto.
`git checkout main`
`git pull origin main`
- 2. Criar uma nova branch de funcionalidade:** Isole seu trabalho criando uma nova ramificação (branch). Uma boa prática é usar o prefixo feature/.
`git checkout -b feature/nome-da-sua-funcionalidade`
- 3. Verificar o estado das alterações:** Após realizar modificações no código, verifique quais arquivos foram alterados e compare as diferenças.
`git status`
`git diff`
- 4. Adicionar arquivos ao Staging Area:** Selecione as alterações que farão parte do seu próximo registro.
`git add .` # Adiciona todas as modificações do diretório atual
`git add caminho/do/arquivo.py` # Adiciona apenas um arquivo específico
- 5. Registrar o Commit:** Salve as alterações com uma mensagem semântica, clara e descritiva.
`git commit -m "feat: adiciona nova funcionalidade ao sistema"`
- 6. Enviar para o Repositório Remoto (Push):** Suba sua branch local para o servidor remoto (ex: GitHub) para a criação posterior do Pull Request. A flag -u (upstream) vincula sua branch local à remota.
`git push -u origin feature/nome-da-sua-funcionalidade`

2. Fluxo Rápido de Correção de Bug (Hotfix Workflow)

O fluxo de hotfix é acionado emergencialmente quando um erro crítico (bug) é descoberto no ambiente de produção. A correção precisa ser aplicada, testada e integrada à branch principal imediatamente, paralisando temporariamente o desenvolvimento de outras funcionalidades.

1. **Salvar o trabalho atual e retornar à branch principal:** Se estiver no meio de um desenvolvimento não commitado, guarde suas alterações provisoriamente com o comando stash e vá para a base estável do projeto.
`git stash`
`git checkout main`
`git pull origin main`
2. **Criar a branch de hotfix:** Crie uma branch derivando diretamente da main mais atualizada. Geralmente utiliza-se o prefixo hotfix/.
`git checkout -b hotfix/descricao-curta-do-erro`
3. **Aplicar a correção e registrar (Commit):** Faça os reparos necessários no código, teste e consolide o commit com urgência.
`git add .`
`git commit -m "fix: corrige falha crítica no fluxo de autenticação"`
4. **Enviar a correção para o repositório remoto:** Suba a branch de hotfix imediatamente para a abertura e aprovação acelerada do Pull Request.
`git push -u origin hotfix/descricao-curta-do-erro`

Dica de Ouro: Após o hotfix ser aprovado e integrado (merge) na main, lembre-se de retornar para sua branch de feature original, recuperar suas alterações pendentes com git stash pop e atualizar sua branch com os novos commits da main (ex: usando git rebase main ou git merge main).

3. Tabela de Referência Rápida de Comandos Adicionais

Comando	Descrição da Ação
git clone <url>	Copia todo um repositório remoto existente para o seu ambiente de desenvolvimento local.
git log --oneline	Exibe o histórico de commits de forma resumida e cronológica na branch atual.
git stash	Armazena temporariamente (numa área de "rascunho") mudanças feitas nos arquivos que ainda não estão prontas para compor um commit.
git stash pop	Recupera as mudanças armazenadas no stash mais recente e as aplica no diretório de trabalho, removendo-as da pilha do stash.
git restore <arquivo>	Descarta modificações locais ainda não adicionadas (unstage) em um arquivo, retornando-o ao estado do último commit.